

Lightbar — Plasma 2350 W Single-Board Research

Goal: Replace the two-board setup (Plasma 2040 + XIAO ESP32-C6) with a single **Pimoroni Plasma 2350 W** and still get Apple Home / smart home integration.

Hardware: Plasma 2350 W

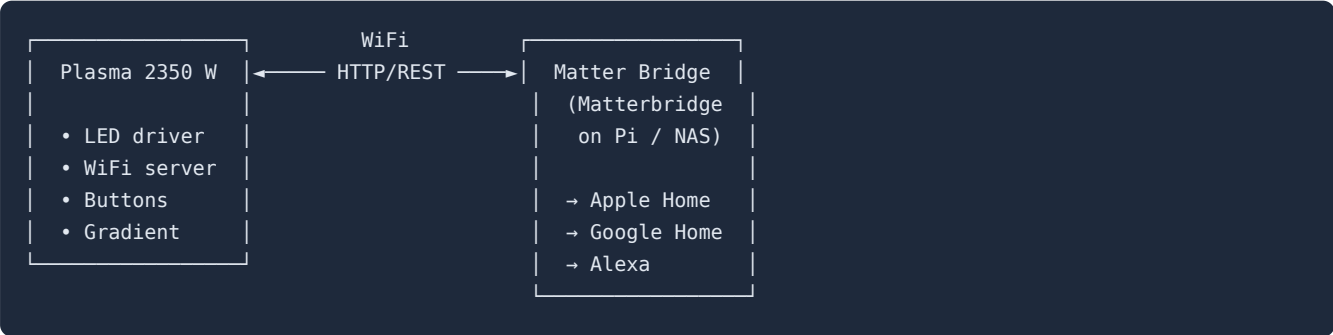
Spec	Detail
MCU	RP2350A (Dual Cortex-M33, 150 MHz, 520 KB SRAM)
Flash	4 MB QSPI
Wireless	Raspberry Pi RM2 module (Infineon CYW43439)
WiFi	802.11 b/g/n 2.4 GHz
Bluetooth	BLE 5.x (via CYW43439)
LED output	WS2812/Neopixel + APA102/Dotstar (screw terminals, PIO-driven)
USB	USB-C (power + programming, 3A max)
Buttons	User button + BOOT (usable as 2nd button) + RESET
Extras	RGB status LED, Qw/ST connector, current sense
Programming	MicroPython (Pimoroni firmware) or C/C++ (Pico SDK)

Key advantage over Plasma 2040: Same form factor, same screw terminals, same PIO LED driving — but adds WiFi + Bluetooth via the RM2 module. The RP2350 is also ~2× faster than the RP2040 with hardware floating point.

Option 2: Matter via WiFi Bridge

Concept

The Plasma 2350 W runs a simple **HTTP/REST API over WiFi**. A separate bridge (running on a Pi, NAS, or any always-on machine) translates between the REST API and the Matter protocol, exposing the light to Apple Home / Google Home / Alexa.



Bridge Options

A) Matterbridge (recommended)

- **What:** Node.js-based Matter plugin manager built on `matter.js`
- **GitHub:** `Luligu/matterbridge` — very active, 4K+ stars
- **How:** Runs on any machine with Node.js (Pi, NAS, Docker, Mac)
- **Plugin:** Write a small plugin that polls the Plasma's REST API and exposes it as a Matter Extended Color Light
- **Pairing:** Pair Matterbridge once → all plugins appear in Apple Home
- **Pros:** Works with ALL ecosystems (Apple, Google, Alexa, HA), lightweight (runs on 512 MB), active community, plugin template available
- **Cons:** Requires a separate always-on device

B) Home Assistant + Matter Server

- **What:** HA discovers the Plasma via REST/MQTT, then exposes it via its built-in Matter server or the Matterbridge HA add-on
- **How:** Add the Plasma as a RESTful light in HA config, then expose to Matter
- **Pros:** If you already run HA, it's just config
- **Cons:** Heavy dependency, overkill if you don't use HA for other things

C) chip-bridge-app (official Matter SDK)

- **What:** The `connectedhomeip` SDK includes a bridge example
- **How:** Compile and run on a Linux host, bridge WiFi devices to Matter
- **Pros:** Official, most correct implementation
- **Cons:** Complex to build, not very user-friendly, heavy dependencies

Plasma Firmware (Option 2)

MicroPython on the Plasma 2350 W:

- Connect to WiFi on boot
- Run a lightweight HTTP server (using `asyncio` + `socket` or `microdot`)
- Expose REST endpoints for light control
- Keep all existing LED driving / gradient / button logic
- mDNS advertisement so the bridge can discover it

REST API:

```
GET /status      → {"on": true, "brightness": 1.0, "colors": [...]}
POST /on         → turn on
POST /off        → turn off
POST /brightness → {"value": 0.8}
POST /color      → {"colors": [{"h":0.6,"s":1,"v":1}, ...]}
POST /gradient   → {"type": "2color", "c1": {...}, "c2": {...}}
```

Verdict: Option 2

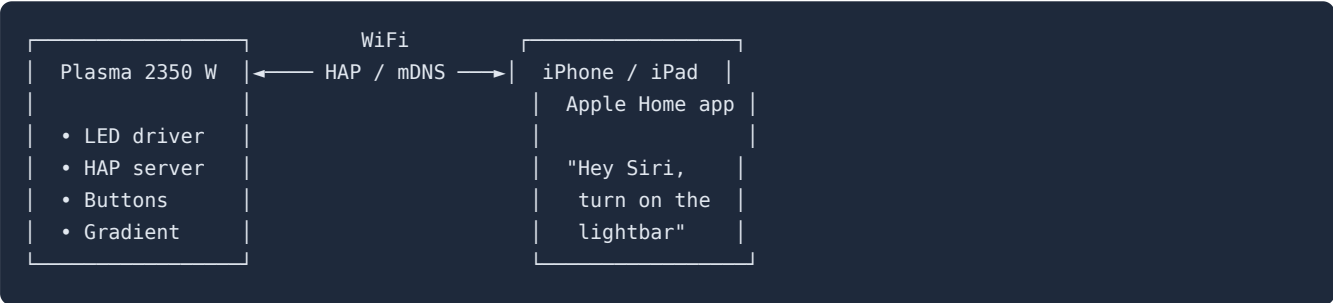
Aspect	Rating
Single board?	☐ Yes
Apple Home?	☐ Yes (via bridge)
Google/Alexa?	☐ Yes (via bridge)

Requires extra hardware?	⚠ Yes — any always-on device for the bridge
Complexity	Medium — firmware is simpler (no UART), but need bridge setup
Reliability	Good — WiFi is proven on Pico W, Matterbridge is mature
Latency	~50-200ms (WiFi round-trip + bridge overhead)

Option 3: Native HomeKit over WiFi (HAP)

Concept

The Plasma 2350 W runs a native **HomeKit Accessory Protocol (HAP)** implementation directly on the board. It appears as a native HomeKit accessory — no bridge, no extra hardware.



What HAP Requires

HomeKit Accessory Protocol over IP (HAP-IP) needs:

1. **mDNS/DNS-SD** — advertising the accessory on the network (`_hap._tcp`)
2. **SRP (Secure Remote Password)** — for pairing (setup code like 123-45-678)
3. **Ed25519** — for long-term identity keys
4. **Curve25519 + HKDF + ChaCha20-Poly1305** — for encrypted sessions
5. **HTTP/1.1** over the encrypted channel — for accessory interaction
6. **HAP JSON data model** — services and characteristics

Feasibility on RP2350

Component	Available?	Notes
WiFi	☐	CYW43439, works great in MicroPython
mDNS	☐	MicroPython has basic mDNS; C SDK has full lwIP mDNS
SRP-6a	⚠	No existing MicroPython lib; needs C module or pure-Python (slow)
Ed25519	⚠	Not in standard MicroPython; needs C module (<code>monocypher</code> or <code>tweetnacl</code>)
X25519/HKDF	⚠	Same — needs C crypto module
ChaCha20-Poly1305	⚠	Same
HTTP server	☐	Trivial in MicroPython
RAM (520 KB)	☐	HAP is lightweight once crypto is native

Flash (4 MB)	☐	Plenty
--------------	--------------------------	--------

Existing Work

- **kevinmcaleer/micropython-matter** — implements SPAKE2+ and crypto primitives as a C module for MicroPython on Pico W. While this targets Matter (not HAP), the crypto approach (C modules compiled into firmware) is the same pattern we'd need for HAP.
- **HAP-python** — full HAP implementation in Python, but requires CPython (cryptography library, asyncio). Too heavy for MicroPython directly, but the protocol logic can be ported.
- **maximkulkin/esp-homekit** — C implementation of HAP for ESP32/ESP8266. The protocol implementation could be ported to Pico SDK (C/C++).
- **HomeSpan** — Arduino HAP library for ESP32. Not directly portable to RP2350 but the protocol logic is well-documented.

Implementation Path

Approach A: MicroPython + C crypto module (recommended)

1. Build a custom MicroPython firmware for the Plasma 2350 W that includes C modules for: Ed25519, X25519, HKDF-SHA512, ChaCha20-Poly1305, SRP-6a
2. Write the HAP protocol layer in pure MicroPython (pairing, sessions, HTTP)
3. Expose the light as a HAP Lightbulb service with HSV color characteristics

Estimated effort: **2-4 weeks** for someone experienced with HAP + embedded crypto. The crypto C modules are the hard part; the protocol layer is well-documented.

Approach B: C/C++ with Pico SDK (alternative)

1. Port `esp-homekit` or write HAP from scratch using Pico SDK + lwIP
2. Use PIO for WS2812 driving (already supported in Pico SDK examples)
3. Everything in C — faster, smaller, but harder to iterate

Estimated effort: **3-6 weeks**. More work but more control and better performance.

Limitations of HAP-only

- **Apple Home only** — HAP doesn't work with Google Home or Alexa
- **No Thread** — WiFi only (but that's fine for this use case)
- **Uncertified** — won't have the official "Works with Apple HomeKit" badge, but non-commercial HAP implementations work fine in practice
- **Pairing limits** — HAP accessories support up to 16 paired controllers

Verdict: Option 3

Aspect	Rating
Single board?	☐ Yes
No extra hardware?	☐ Yes — truly standalone
Apple Home?	☐ Yes (native)
Google/Alexa?	☐ No

Complexity	High — need to implement/port HAP crypto + protocol
Reliability	Very good once working — direct connection, no bridge
Latency	Low (~20-50ms, direct WiFi, no bridge hop)
Development effort	High (2-6 weeks depending on approach)

Option 1 (Bonus): Plasma 2350 W + ESP32-C6

Concept

Keep the ESP32-C6 for Matter/Thread (proven, supported by Espressif's Matter SDK), but replace the Plasma 2040 with the Plasma 2350 W.



What Changes

Almost nothing in code:

- The Plasma 2350 W has the same QW/ST connector on the same pins
- Same PIO for WS2812 driving
- Same button pins, same status LED, same current sense
- MicroPython firmware needs to be the Pimoroni **2350** build instead of 2040
- UART pins might differ slightly (need to verify Plasma 2350 W pinout)
- The WiFi/BT capability goes unused (but doesn't hurt)

What You Gain

- RP2350 is ~2× faster than RP2040 (better gradient calculations)
- Hardware floating point (gradient math is all float)
- 520 KB SRAM vs 264 KB (room for bigger LED counts or effects)
- Current A4 stepping (fixes RP2350-E9 erratum)
- Future option: use WiFi for OTA updates or secondary control

Verdict: Option 1

Aspect	Rating
Single board?	☐ No — still two boards
Apple Home?	☐ Yes (native Matter/Thread)
Google/Alexa?	☐ Yes

Complexity	Very low — minimal code changes
Reliability	Highest — proven ESP32 Matter stack
Latency	Lowest — Thread is optimized for this
Development effort	~1 day (pin verification + firmware swap)

Comparison Matrix

	Option 1 (2 boards)	Option 2 (WiFi bridge)	Option 3 (Native HAP)
Boards	Plasma 2350 W + ESP32-C6	Plasma 2350 W only	Plasma 2350 W only
Extra hardware	ESP32-C6 (~\$5)	Bridge device (Pi, etc.)	None
Apple Home	☐ Native Matter	☐ Via bridge	☐ Native HAP
Google Home	☐	☐	☐
Amazon Alexa	☐	☐	☐
Protocol	Matter/Thread	Matter/WiFi (bridged)	HomeKit/WiFi
Latency	~10-30ms	~50-200ms	~20-50ms
Dev effort	1 day	1 week	2-6 weeks
Firmware lang	MicroPython + ESP-IDF	MicroPython	MicroPython + C
Reliability	Very high	High (depends on bridge)	High (once stable)
Standalone	No (2 boards)	No (needs bridge)	Yes
OTA updates	Via ESP32 WiFi	Via WiFi	Via WiFi

Recommendation

If **Apple Home only** is acceptable and you want true single-board: → **Option 3 (Native HAP)** — most elegant, no extra hardware, but significant development effort for the crypto/protocol layer.

If **multi-ecosystem** matters (Apple + Google + Alexa): → **Option 2 (WiFi bridge via Matterbridge)** — single board for the light, bridge runs on any existing device. Easiest to implement.

If **reliability and speed** are top priority: → **Option 1 (Plasma 2350 W + ESP32-C6)** — minimal changes, proven Matter stack, but still two boards.

Research completed March 2026